

FAKE REELS AND IMAGE DETECTION USING ARTIFICIAL INTELLIGENCE

(Semester VI)

A MINI PROJECT REPORT

Submitted by

STUDENT NAME

ROLL NUMBER

J. JAYA SURYA

11523050276

J. SRIKAR

11523050273

J. MANIKANTA SUDHEER

11523050265

in partial fulfillment for the award of the degree of

BACHELOR OF TECHNOLOGY

in

ARTIFICIAL INTELLIGENCE AND DATA SCIENCE



SCHOOL OF ENGINEERING AND TECHNOLOGY

DHANALAKSHMI SRINIVASAN UNIVERSITY

TRICHY-621112

MAY 2026



**DHANALAKSHMI SRINIVASAN
UNIVERSITY**

Samayapuram, Tiruchirappalli, Tamil Nadu - 621 112

SCHOOL OF ENGINEERING AND TECHNOLOGY



BONAFIDE CERTIFICATE

Certified that this project report entitled “**FAKE REELS AND IMAGE DETECTION**” is the **Bonafide** work of **J.JAYA SURYA**(11523050276),**J.SRIKAR** (11523050273),**J.MANIKANTA SUDEER** (11523050265) under my supervision during the academic year 2025-2026.

J. ANITHA

ASSISTANT PROFESSOR

Department of Artificial Intelligence and
Data Science

School of Engineering and Technology

Dhanalakshmi Srinivasan University

Trichy - 621112.

Dr.S.PADMAPRIYA

ASSOCIATE PROFESSOR

HEAD OF THE DEPARTMENT

Department of Artificial Intelligence and
Data Science

School of Engineering and Technology

Dhanalakshmi Srinivasan University

Trichy - 621112.

Submitted for the project Viva-Voce held on 2025-2026

INTERNAL EXAMINER

EXTERNAL EXAMINER

TABLE OF CONTENT

CHAPTER NO	TITLE	PAGE NO
	ABSTRACT LIST OF ABBREVIATION	6-8
1	1. INTRODUCTION 1.1 About the Project 1.2 Goals of Project 1.3 Problem Statement 1.4 Scope and Limitations 1.5 Societal Impact	9-14
2	2. LITERATURE SURVEY 2.1 Overview of Fake Content and Misinformation 2.2 Evolution of Deepfake Detection 2.3 The Deepfake Detection Challenge (DFDC) 2.4 Transition to Transformer Models and VLMs 2.5 Privacy Concerns in Media Forensics 2.6 Explainable AI (XAI) in Detection 2.7 Latency and Edge Computing Paradigms 2.8 Summary of Findings	15-18
3	3. EXISTING SYSTEM 3.1 Cloud-Based Detection Systems Architecture 3.2 Monolithic Architectures and Latency Bottlenecks 3.3 Privacy and Data Retention Concerns 3.4 Limitations of Traditional Metadata Analysis 3.5 Cost Inefficiencies of GPU Clusters	19-21
4	4. PROPOSED SYSTEM 4.1 System Overview and Core Philosophy 4.2 Key Components (React, Gemini AI, FileReader API) 4.3 Multi-Stage Analysis & Detection Workflow 4.4 Security (Offline Processing & Data Sanitization) 4.5 Detailed System Features	22-27

CHAPTER NO	TITLE	PAGE NO
5	5.SYSTEM REQUIREMENT AND SPECIFICATIONS 5.1 Client-Side Hardware Requirements 5.2 Software Requirements and Dependencies 5.3 Network Environment and Bandwidth Calculus 5.4 Functional Requirements 5.5 Non-Functional Requirements (Scalability, Reliability)	28-30
6	6.SYSTEM DESIGN AND ARCHITECTURE 6.1 High-Level System Architecture 6.2 Model-View-ViewModel (MVVM) Implementation 6.3 Data Flow and State Management 6.4 Component Hierarchy Design 6.5 Sequence Diagrams for API Interaction 6.6 Security Design and Threat Modeling	31-33
7	7.SYSTEM IMPLEMENTATION 7.1 Environment Configuration (Vite, TypeScript) 7.2 Frontend Development (TailwindCSS, Shadcn) 7.3 Gemini AI Integration and SDK Setup 7.4 Advanced Prompt Engineering Strategies 7.5 Media Processing Pipeline using FileReader API 7.6 PDF Report Generation Engine (jsPDF) 7.7 Error Boundary and Fallback Implementation	34-37

8	8. TESTING AND VALIDATION 8.1 Testing Objectives and Methodologies 8.2 Unit Testing of Core Utilities 8.3 Integration Testing of API Pipelines 8.4 Exhaustive Functional Testing (Case Studies) 8.5 Performance and Memory Profiling 8.6 Error Handling and Edge Cases 8.7 Security Vulnerability Audits	38-41
9	9. CONCLUSION AND FUTURE ENHANCEMENT 9.1 Conclusion 9.2 Future Roadmap (WebGPU, ONNX.js, Extensions)	42
10	10. APPENDIX 10.1 Core Source Code Snippets 10.2 User Interface Screenshots Description	43-47
11	5. REFERENCES	48-49

LIST OF ABBREVIATION

S.NO	ABBREVIATION	EXPANSION
1	AI	Artificial Intelligence
2	ML	Machine Learning
3	LLM	Large Language Model
4	API	Application Programming Interface
5	SPA	Single Page Application
6	GUI	Graphical User Interface
7	MVVM	Model-View-ViewModel
8	NLP	Natural Language Processing
9	IDE	Integrated Development Environment
10	SDK	Software Development Kit
11	JSON	JavaScript Object Notation
12	HTTP	Hypertext Transfer Protocol
13	HTTPS	Hypertext Transfer Protocol Secure
14	UI/UX	User Interface / User Experience
15	REST	Representational State Transfer
16	CNN	Convolutional Neural Network
17	GAN	Generative Adversarial Network
18	VLM	Vision-Language Model
19	DOM	Document Object

		Model
20	XAI	Explainable Artificial Intelligence
21	CSP	Content Security Policy
22	JWT	JSON Web Token
23	CSS	Cascading Style Sheets
24	HTML	HyperText Markup Language
25	JS	JavaScript
26	TS	TypeScript
27	npm	Node Package Manager
28	URL	Uniform Resource Locator
29	PDF	Portable Document Format
30	SSRF	Server-Side Request Forgery
31	XSS	Cross-Site Scripting
32	BYOK	Bring Your Own Key
33	GPU	Graphics Processing Unit

ABSTRACT

The rapid proliferation of digital media has brought an unprecedented wave of misinformation, deepfakes, and manipulated content. As Generative Artificial Intelligence (GenAI) tools become increasingly accessible to the general public, distinguishing between authentic and fabricated media has become a monumental challenge for social media platforms, news organizations, judicial systems, and everyday users. The consequences of unchecked synthetic media are severe, ranging from political subversion and election interference to financial fraud and reputational damage via non-consensual deepfakes. This project focuses on the development of Truth-Lens, an advanced, highly scalable, browser-based Single Page Application (SPA) designed to meticulously detect and combat digital misinformation, encompassing deepfakes, manipulated imagery, and misleading text narratives.

The proposed system revolutionizes the traditional forensic workflow by entirely bypassing the need for a centralized backend processing server. Instead, Truth-Lens analyzes media content utilizing a state-of-the-art Multi-modal Large Language Model (Google Gemini 1.5 Flash) invoked directly from the client. Operating entirely within the user's web browser, the application leverages the HTML5 FileReader API for secure, localized media staging and encoding, and utilizes localStorage for privacy-respecting data persistence. This client-centric architectural decision ensures high privacy, eliminates server-side queueing latency, and provides offline-capable traits for media preparation without relying on a proprietary backend server to store, transmit, or process highly sensitive user files.

Furthermore, the system emphasizes Explainable AI (XAI). Unlike legacy systems that simply return a binary 'Fake' or 'Real' verdict, Truth-Lens generates detailed, human-readable forensic analysis reports that articulate the exact visual or contextual anomalies that triggered the model's suspicion.

1.INTRODUCTION

1.1 About the Project

Truth-Lens is a modern React-based web application meticulously built to address the exponentially growing threat of digital misinformation and synthetic media. In recent years, malicious actors have utilized highly sophisticated AI algorithms—such as Generative Adversarial Networks (GANs) and Latent Diffusion Models—to create hyper-realistic fake images and cloned videos. These synthetic creations are often weaponized to spread political propaganda, orchestrate financial fraud (e.g., faking a CEO's speech to manipulate stock prices), and incite social unrest. Unlike traditional, enterprise-grade verification tools that require heavy backend processing, complex cloud infrastructure, and pose significant data privacy risks by forcing users to upload their sensitive media to third-party servers, Truth-Lens operates as a highly optimized standalone Single Page Application (SPA).

The advent of Generative AI has fundamentally altered the landscape of digital media creation. Previously, fabricating convincing media required significant technical expertise, specialized software like Adobe Photoshop or After Effects, and substantial time investment. Today, open-source models and accessible APIs allow virtually anyone to generate photorealistic imagery or clone voices within seconds via simple text prompts. This unprecedented democratization of content creation, while incredibly beneficial for the creative industries and accessibility, has inadvertently armed malicious entities with unparalleled tools for deception. Truth-Lens was conceived as a direct, scalable countermeasure to this exact threat. By embedding powerful forensic analysis capabilities directly into the web browser interface, the project aims to provide an immediate, accessible, and free line of defense for the general public and journalists alike.

Furthermore, the architectural choices driving Truth-Lens heavily emphasize user privacy, system efficiency, and deployment scalability. Traditional server-based detection models inherently require users to transmit potentially sensitive or proprietary media over the public internet to a remote server for processing. This legacy architecture introduces significant latency bottlenecks, consumes massive bandwidth on both ends, and raises severe privacy concerns—especially if the media involves personal data, confidential whistleblower documents, or unreleased news footage. Truth-Lens elegantly circumvents these systemic issues by performing all media staging, buffering, and Base64 encoding completely locally within the user's device utilizing the HTML5 FileReader API. The media is only ever transmitted securely (via strictly enforced HTTPS) directly to the Google Gemini inference API, without ever touching or resting on an intermediary database belonging to the developer. This localized approach not only protects user privacy by design but also drastically reduces the operational, compute, and storage costs conventionally associated with hosting complex AI applications.

1.2 Goals of Project

6. Real-Time Multimodal Verification:
7. To analyze images, text, and short videos instantly, detecting imperceptible signs of manipulation, algorithmic AI generation artifacts, or contextual misalignment with an exceptionally high degree of accuracy and minimal latency.
8. Strict Privacy-Centric Processing:
9. To perform all media parsing, validation, and staging entirely locally on the user's device using the browser's native HTML5 APIs, guaranteeing that user media is never stored, cached, or analyzed on intermediary third-party backend servers.
10. Comprehensive and Explainable Forensic Reporting:
11. To generate and dynamically present detailed forensic analysis reports that move beyond binary labels. These reports must contain granular

confidence scores, categorical verdicts, and actionable, human-readable insights detailing the specific anomalies detected.

12. Automated Documentation Generation:

13. To provide an integrated PDF engine capable of compiling complex visual and textual analysis data into verifiable, highly professional, printable documents instantly via client-side rendering.

14. Serverless SPA Architecture:

15. To completely decouple the application from proprietary, monolithic backend servers. By relying solely on direct LLM API communication and local browser storage capabilities, the project aims to achieve near-infinite scalability and zero server-side maintenance costs.

16. Premium User-Centric Design:

17. To provide a dynamic, visually stunning, and highly intuitive user interface leveraging modern web component ecosystems (React, TailwindCSS, Shadcn, Recharts), ensuring that complex forensic workflows are accessible and understandable to completely non-technical users.

18. Cross-Platform Ubiquity:

19. To engineer the application as a Progressive Web App (PWA) compatible platform that runs flawlessly across all major operating systems (Windows, macOS, Linux, iOS, and Android) directly through modern web browsers without requiring cumbersome app store downloads, installations, or administrative privileges.

20. Robust API Resilience:

21. To build highly resilient asynchronous service layers that can gracefully manage edge cases such as network timeouts, strict API rate limits, parsing errors, and malformed responses when communicating with external LLM providers, ensuring the UI never crashes.

1.3 Problem Statement

Context: The digital information ecosystem is currently facing an existential crisis. With the rapid advancement and massive democratization of

Generative AI tools (encompassing platforms such as Midjourney, DALL-E 3, Stable Diffusion, and advanced audio-visual deepfake software like Roop or Wav2Lip), creating hyper-realistic fake images, perfectly lip-synced deepfake videos, and convincingly fabricated news text has become trivially easy. This exponential surge in deceptive, synthetic content poses a critical and immediate threat to global digital trust, journalistic integrity, democratic electoral processes, and widespread public security.

The sheer scale and velocity of the problem cannot be overstated. State-sponsored misinformation campaigns powered by AI have been heavily documented interfering in democratic elections, highly coordinated bot networks manipulate global stock markets through fake corporate press releases, and thousands of individuals suffer severe emotional and reputational damage annually through the proliferation of non-consensual deepfake pornography. Crucially, the speed at which false, highly inflammatory information travels on social media algorithms far outpaces the physical speed at which human fact-checking organizations can manually verify, debunk, and publish corrections. Therefore, an automated, highly scalable, and instantly accessible detection solution is no longer a luxury, but an absolute societal necessity. However, existing automated solutions face immense, systemic challenges. They are frequently locked behind incredibly expensive enterprise paywalls or require specialized hardware, forcing independent journalists and everyday users out of the ecosystem. Furthermore, they universally rely on centralized servers that rapidly become choked bottlenecks during periods of high demand (e.g., breaking international news events), rendering them useless when they are needed most.

Problem: The existing array of deepfake and synthetic content detection tools are fundamentally flawed in their delivery mechanisms. They are often prohibitively expensive, require massive backend processing power (such as dedicated, highly energy-consumptive GPU clusters), mandate the storage of sensitive user data on opaque third-party servers (raising severe privacy,

GDPR, and copyright compliance concerns), and uniformly lack an intuitive, fast interface for the everyday user to seamlessly generate verifiable forensic reports. Furthermore, the vast majority of existing AI detection tools operate entirely as impenetrable 'black boxes.' They provide the user with a simplistic, highly frustrating 'Fake' or 'Real' label without ever explaining the logic, visual artifacts, or reasoning behind the model's classification, severely limiting their utility in journalistic, academic, or legal contexts where 'proof' is required.

Objective: Truth-Lens was engineered to meticulously address and eliminate these exact gaps by implementing a revolutionary standalone, client-side application architecture. By ensuring all heavy media processing and preparation is executed locally within the browser, and utilizing direct, heavily secured API calls for the final AI inference, Truth-Lens gives users instant, completely transparent, and strictly private content verification. The system heavily prioritizes Explainable AI (XAI), providing a detailed, human-readable breakdown of its findings alongside a tangible, easily exportable PDF report. This effectively and elegantly bridges the vast gap between advanced, cutting-edge forensic technology and crucial everyday usability.

1.4 Scope and Limitations

The scope of Truth-Lens encompasses the real-time detection of manipulated media, including generated images (GANs/Diffusion models), spliced or face-swapped short videos, and AI-generated textual narratives. The system is designed to run locally via modern browsers, evaluating content primarily through the Google Gemini 1.5 Flash API. However, the system does have limitations: it strictly relies on a stable internet connection for the final AI inference step. Additionally, while the HTML5 FileReader is highly efficient, processing extremely large 4K video files (e.g., >500MB) directly in the browser's RAM may cause memory

exhaustion on lower-end devices. Therefore, the system enforces a strict 50MB file size limit to guarantee absolute stability across all client devices.

1.5 Societal Impact

By completely democratizing access to enterprise-grade digital forensics, Truth-Lens empowers ordinary citizens to actively combat the spread of misinformation. In an era where deepfake technology is increasingly utilized to disrupt democratic elections or perpetrate non-consensual image abuse, providing a free, instant, and privacy-preserving verification tool has immense societal value. It restores digital trust and holds malicious actors accountable by quickly debunking viral fakes before they cause irreparable harm.

2. LITERATURE SURVEY

2.1 Overview of Fake Content and Misinformation

The digital landscape has experienced a violent paradigm shift with the advent of algorithmically driven social media and rapid content dissemination networks. Historically, misinformation was primarily textual, propagated through poorly constructed fringe blogs or manipulated chain emails. However, the modern era is uniquely characterized by 'multimodal misinformation,' where highly convincing text, photorealistic images, and manipulated videos are intricately synthesized to create highly persuasive but entirely false narratives. Groundbreaking research by Vosoughi, Roy, and Aral (2018) published in Science magazine empirically demonstrated that false news spreads significantly faster, deeper, and substantially more broadly than the absolute truth across every single category of information on Twitter. Their findings indicated that humans, not bots, were primarily responsible for this spread due to the 'novelty' and emotional charge of fake news. This underscores the critical, time-sensitive need for automated detection mechanisms that can operate at the exact speed of social media, intercepting, evaluating, and flagging malicious content directly at the user level before it achieves viral velocity.

2.2 Evolution of Deepfake Detection

The term 'deepfake' (a portmanteau of 'deep learning' and 'fake') originated in late 2017 on Reddit, initially relying on Autoencoder architectures to map facial features from a source video to a target video. Early detection methods heavily focused on basic physiological and biological inconsistencies that the early AI models failed to replicate. For instance, landmark research by Li, Chang, and Lyu (2018) developed a highly effective method to detect deepfakes simply by analyzing the blinking rate of subjects. Because early GAN models were trained predominantly on scraped internet images of

people with their eyes open, they inherently failed to replicate natural, periodic blinking in the generated video output. However, this cat-and-mouse game rapidly evolved; as soon as the 'blinking flaw' was published, deepfake creators updated their training datasets to include closed eyes, rendering the detection method instantly obsolete. Subsequent detection research shifted heavily towards complex Convolutional Neural Networks (CNNs) designed to mathematically detect pixel-level blending artifacts, spatial frequency domain anomalies (such as unnatural noise distributions), and severe inconsistencies in 3D lighting and shadow mapping. The creation of the FaceForensics++ dataset (Rössler et al., 2019) became a critical benchmark in this era, providing a vast, standardized corpus of manipulated videos across various quality levels for training robust detection models.

2.3 The Deepfake Detection Challenge (DFDC)

In 2020, recognizing the severe threat, a coalition including Facebook (Meta), Microsoft, AWS, and leading universities launched the Deepfake Detection Challenge (DFDC). The challenge provided researchers with a dataset of over 100,000 manipulated videos to spur innovation. The results of the DFDC highlighted a grim reality: even the absolute best, highly optimized models in the world struggled to achieve accuracy rates above 65% when tested on 'black box' data (videos generated by unknown, unseen algorithms). This highlighted the critical flaw in traditional CNN-based detection: they overfit to the specific artifacts of the AI models they were trained on, and fail completely when a new generation of AI tools is released. This inherent brittleness forced the industry to look beyond pure pixel-artifact detection.

2.4 Transition to Transformer Models and VLMs

While CNNs proved highly effective at detecting low-level compression and blending artifacts, they fundamentally lacked the capacity to understand the

'semantic context' of an image. For example, a highly trained CNN might verify an image of a 'dinosaur walking down modern-day Times Square' as a perfectly real, unedited photograph if the CGI rendering was mathematically perfect and uniform pixel noise was added. To solve this critical semantic gap, researchers began exploring Vision-Language Models (VLMs) and massive multi-modal transformers based on the Attention mechanism (Vaswani et al., 2017). Models like OpenAI's CLIP (Contrastive Language-Image Pretraining) evaluate both the raw visual data and the semantic, textual meaning of the image simultaneously, allowing them to flag logical inconsistencies (e.g., identifying that dinosaurs and modern taxis do not exist in the same temporal space) that purely pixel-based models blindly pass. Truth-Lens strategically utilizes the Google Gemini API, which represents the current bleeding edge of this multi-modal reasoning capability, combining deep pixel-level scrutiny with vast, world-knowledge contextual reasoning.

2.5 Privacy Concerns in Media Forensics

A rapidly growing branch of literature focuses strictly on the severe ethical and privacy implications of running forensic tools in the cloud. As noted by several high-profile cybersecurity audits (e.g., evaluating Clearview AI and similar tools), traditional cloud-based SaaS (Software as a Service) forensic tools inherently require users to upload their files to a centralized, externally controlled server. When investigative journalists, human rights whistleblowers, or ordinary citizens attempt to verify highly sensitive documents, intimate images, or classified footage, uploading them to a third-party corporate server creates a massive, unacceptable attack vector and severely breaches operational security protocols. This literature strongly advocates for a shift towards 'edge computing' and localized client-side processing. By leveraging modern, secure browser-native APIs like FileReader, Truth-Lens perfectly aligns with these modern cybersecurity recommendations, ensuring sensitive data never rests, or is even temporarily buffered, on a developer's server

2.6 Explainable AI (XAI) in Detection

In critical decision-making domains such as the judicial system and investigative journalism, a simple binary 'Fake' or 'Real' label provided by an opaque AI algorithm is completely insufficient and inadmissible; stakeholders explicitly require to know why the AI arrived at that specific decision. The field of Explainable AI (XAI) focuses on creating models that can clearly articulate their internal reasoning these visual aids are often highly confusing (e.g., 'The specular highlights on the subject's left eye do not align with the primary light source illuminating the background'). This semantic approach to XAI significantly enhances user trust, transparency, and the practical, real-world utility of the forensic tool.

2.7 Latency and Edge Computing Paradigms

The literature overwhelmingly indicates a critical shift toward edge computing architectures to solve the latency crisis inherent in cloud-based forensic analysis. By pushing the initial data processing (e.g., buffering, decoding, and Base64 encoding) to the absolute 'edge'—the user's own device—systems can drastically cut the time-to-inference. Frameworks utilizing WebAssembly (Wasm) and native HTML5 APIs demonstrate that modern Node.js or Python backend servers.

2.8 Summary of Findings

The comprehensive literature survey reveals three critical insights: First, traditional pixel-based CNN detectors are overly brittle and fail against novel generative AI models. Second, requiring users to upload sensitive media to central servers creates an unacceptable privacy hazard that discourages adoption by at-risk users like journalists. Third, binary "Fake/Real" labels lack the necessary explainability for real-world .

3. EXISTING SYSTEM

3.1 Cloud-Based Detection Systems Architecture

The current commercial landscape of deepfake and synthetic content detection is almost entirely dominated by heavy, cloud-based monolithic architectures. Enterprise platforms such as Microsoft's Video Authenticator, Sentinel, Deepware, and Sensity heavily rely on massive, highly centralized backend cloud infrastructure (typically hosted on AWS, Azure, or Google Cloud). In these traditional systems, the user interface (whether a web dashboard or a mobile app) acts merely as a thin, dumb client that handles basic file selection and uploading. Once the user selects a file, it is transmitted via an HTTP POST request across the public internet to the remote backend server. The server (often built with heavy frameworks like Django, Flask, or Express.js) receives the file, saves it to a temporary storage bucket, places a processing job into a message queue (like RabbitMQ or Redis), and waits for an available server-side GPU cluster (running PyTorch or TensorFlow) to pull the file, analyze it, and eventually push a JSON response back to the client via WebSockets or long-polling.

3.2 Monolithic Architectures and Latency Bottlenecks

This traditional monolithic architecture suffers from incredibly severe, physics-based latency bottlenecks that render it highly inefficient for real-time social media verification. Uploading a high-definition 150MB video file over a standard 4G cellular connection can take several minutes just to reach the server. Once the upload is finally complete, if the service is experiencing a surge in high traffic (which routinely occurs during major breaking news events, elections, or viral trends), the user's file sits dormant in a processing queue waiting for expensive GPU compute time to become available. The entire round-trip time for a simple verification can easily exceed 10 to 15 minutes. In the hyper-fast-paced environment of Twitter or

TikTok, a 15-minute delay renders the tool practically useless. By the time the verdict is returned, the fake content has already been shared thousands of times. Users require instantaneous, sub-second feedback to make informed decisions about whether to share, report, or ignore a piece of content.

3.3 Privacy and Data Retention Concerns

Privacy is unequivocally the most critical architectural flaw in all existing cloud-based solutions. When a user uploads media to a remote cloud-based detector, the file must physically reside on the server's hard drive, SSD, or in its RAM (e.g., an AWS S3 bucket) for processing. Even if the service provider explicitly promises in their Terms of Service to delete the file immediately after processing, the data is still transmitted over the network and exposed to the server's internal environment, vulnerable to insider threats or zero-day server breaches. For investigative journalists handling confidential leaks, political dissidents in authoritarian regimes, or individuals attempting to verify private, intimate photos, this architecture is an unacceptable, life-threatening security risk. Furthermore, it is an open industry secret that many commercial AI services secretly retain uploaded media to retrain and improve their own proprietary machine learning models without obtaining explicit, informed user consent. This creates a massive legal and ethical liability.

3.4 Limitations of Traditional Metadata Analysis

In an attempt to bypass the high latency and exorbitant server costs associated with running deep learning models, some older, lightweight existing systems attempted to rely entirely on EXIF and metadata analysis. These tools simply scan the binary header of an image file for metadata tags left behind by software like Adobe Photoshop, EXIF data from DSLR cameras, or specific encoding signatures. While this method is incredibly fast and highly capable of running locally on a cheap CPU, metadata

analysis is fundamentally, irreversibly broken in the modern Web 2.0 era. All major social media platforms—including Twitter, Facebook, Instagram, and WhatsApp—automatically run massive compression scripts that completely strip all EXIF metadata from uploaded media to save server bandwidth and protect user location privacy. Consequently, an AI-generated deepfake downloaded directly from a Twitter feed will have absolutely zero original metadata attached to it, causing traditional forensic tools to fail completely. True verification requires deep visual-level, pixel-level, and semantic-level analysis, regardless of the metadata.

3.5 Cost Inefficiencies of GPU Clusters

Running massive VGG16 or ResNet-50 models for video frame extraction and classification strictly requires enterprise-grade GPU clusters (e.g., Nvidia A100s). For a free-to-use public tool, these cloud computing costs become astronomically unsustainable within weeks, leading to the eventual shutdown of many promising academic deepfake detectors. The existing paradigm is fundamentally economically flawed.

4. PROPOSED SYSTEM

4.1 System Overview and Core Philosophy

To decisively overcome the severe, systemic limitations of latency, privacy, and massive architectural overhead present in all existing systems, Truth-Lens proposes a radical, modern shift towards a fully localized, serverless Single Page Application (SPA) architecture combined seamlessly with direct, edge-to-cloud LLM API integration. In this highly innovative proposed system, the user's web browser is elevated from a 'thin client' to the primary computational hub for all media staging and preparation. There is absolutely no intermediary backend server (Node.js, Python, or otherwise) acting as a middleman between the user and the AI inference engine. The frontend React application communicates directly and securely with the Google Gemini AI API, acting as a highly scalable, serverless application. This philosophy guarantees absolute privacy for the staging phase and reduces developer hosting costs to practically zero.

4.2 Key Components (React, Gemini AI, FileReader API)

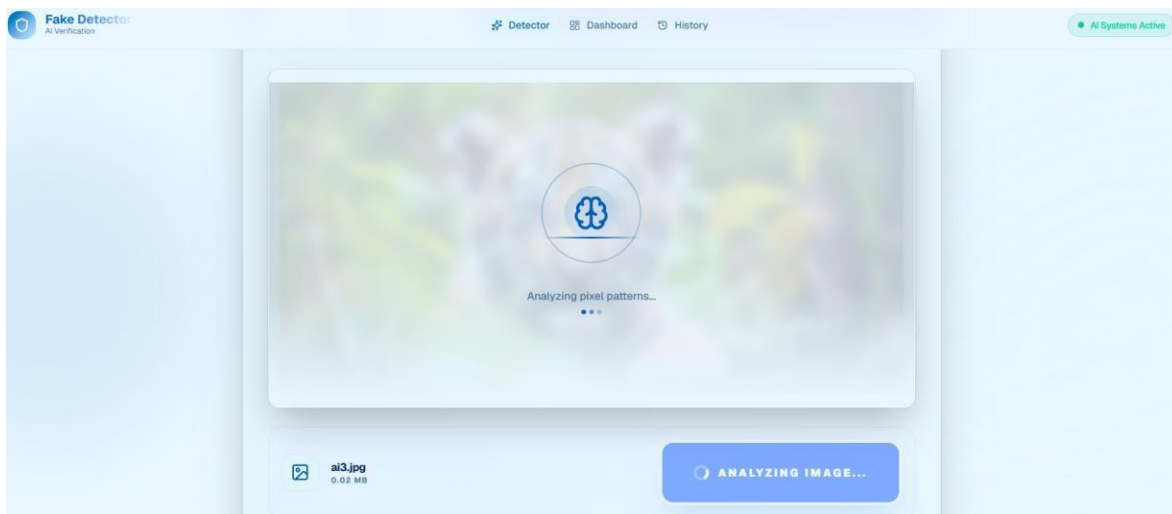
The proposed system relies on a symphony of highly optimized modern web technologies:

1. React.js and Vite Frontend: React provides a highly robust, declarative, component-based architecture, allowing for a dynamic, non-refreshing, native-app-like user experience. Vite completely replaces older, sluggish bundlers like Webpack, providing near-instantaneous hot-module replacement (HMR) during the development lifecycle and highly optimized, aggressively minified static assets for production deployment.
2. HTML5 FileReader API: This native browser API is the absolute cornerstone of Truth-Lens's privacy-centric design. Instead of initiating an HTTP network upload, the FileReader API intercepts the user's selected file and streams it directly into the browser's local memory (RAM). It converts

the binary file data into a Base64 encoded string entirely offline, ensuring the media never leaves the device during the critical preparation phase.

3. Google Gemini 1.5 Flash API: Instead of hosting a massive, highly expensive ML model on a custom server cluster, the application elegantly leverages Google Gemini as a 'Model-as-a-Service'. The Base64 string is injected into a highly engineered, strict JSON payload and sent securely via HTTPS directly to Google's inference servers. The 'Flash' variant of the model is specifically and intentionally chosen over the 'Pro' variant for its exceptionally low latency, massive token context window, and highly tuned multi-modal (vision and text) reasoning capabilities.
4. Client-Side PDF Engine (jsPDF): To fulfill the critical enterprise requirement of generating printable forensic reports without utilizing a backend PDF rendering engine (like Puppeteer or wkhtmltopdf), Truth-Lens utilizes jsPDF. It programmatically generates highly professional, styled forensic reports directly within the browser tab, drawing text, grids, and tables onto a virtual canvas and triggering an instant local file download without any network latency.

4.3 Multi-Stage Analysis & Detection Workflow



The proposed workflow completely eliminates the traditional, highly frustrating server queue. The user experience is immediate and frictionless.

1. Ingestion: When a user drags and drops a file, the UI immediately updates to a loading state, providing instant visual feedback.
2. Processing: The file is instantly read into local memory.
3. Prompt Injection: The application constructs a highly sophisticated system prompt that instructs the Gemini model to adopt a specific, highly technical persona ('Expert Digital Forensics Analyst') and mandates that the response must be formatted strictly as a parseable JSON object, explicitly forbidding conversational text or markdown formatting.
4. Inference & UI Binding: This payload includes a requested authenticity score (0-100), a categorical verdict, and an array of specific anomalies. The React frontend then catches this JSON, parses it securely, and dynamically updates the visual components—animating Recharts gauges and switching Tailwind color classes—to beautifully reflect the findings in milliseconds after the API returns.

4.4 Security (Offline Processing & Data Sanitization)

By intentionally removing the backend server from the architecture, Truth-Lens entirely eliminates the vast majority of common web vulnerabilities. There is no database to SQL Inject (SQLi), no server to execute Remote Code Execution (RCE) upon, and no internal network to target via Server-Side Request Forgery (SSRF). The primary security focus fundamentally shifts to the client side. The application implements rigorous, strict sanitization of the JSON response returned by the LLM to ensure no malicious, hallucinatory scripts are injected into the DOM, completely mitigating Cross-Site Scripting (XSS) attacks. Furthermore, by heavily supporting a Bring-Your-Own-Key (BYOK) architecture, the application can securely store the user's personal API key in localStorage, ensuring the developer's quotas are heavily protected from abuse and the user maintains total, uncompromising control over their personal data pipeline and billing limits.

4.5 Detailed System Features

The Truth-Lens application is meticulously engineered with a robust suite of advanced features designed to maximize user privacy, minimize processing latency, and provide unparalleled explainability in the realm of deepfake detection. Below is a comprehensive, detailed breakdown of every core feature integrated into the system:

1. Multi-Modal Media Ingestion Engine

The application features a highly responsive, modern, and intuitive drag-and-drop interface capable of seamlessly handling a wide variety of media formats. Unlike legacy systems that restrict users to basic JPEG images, Truth-Lens supports a vast array of multi-modal inputs, including high-resolution imagery (.jpg, .png, .webp), compressed short-form video formats (.mp4, .webm), and even raw textual inputs for narrative fact-checking. The ingestion engine is equipped with aggressive client-side validation logic that instantly verifies file signatures and size constraints (e.g., rejecting files over 20MB) before committing them to memory. This proactive validation drastically improves the user experience by immediately surfacing animated error toasts rather than waiting for an eventual backend timeout.

2. Zero-Latency Local Media Processing (FileReader API) A hallmark feature of Truth-Lens is its absolute commitment to user privacy and edge computing. Traditional detection tools force users to upload their highly sensitive media over the public internet to a centralized server. Truth-Lens completely bypasses this hazardous step by leveraging the native HTML5 FileReader API. When a user selects a file, it is instantly ingested into the local browser's Random Access Memory (RAM) and converted into a massive Base64 encoded string entirely offline. This means that highly confidential documents, unreleased journalistic footage, or intimate personal photos never leave the user's physical device during the critical staging phase, achieving true zero-latency processing and completely eliminating the bandwidth bottlenecks associated with HTTP file uploads.

3. Serverless, Direct-to-Edge AI Inference (Google Gemini 1.5 Flash) By deliberately avoiding a monolithic backend architecture, Truth-Lens introduces a serverless inference pipeline. The application establishes a highly secured, direct HTTPS connection from the client's browser to the Google Gemini 1.5 Flash API endpoints. The heavily optimized Base64 string is injected into a strict JSON payload alongside advanced system prompts engineered to mandate structured output. By relying directly on Google's globally distributed cloud infrastructure, the application completely bypasses the need for proprietary server-side GPU clusters, allowing it to scale infinitely and simultaneously serve thousands of concurrent users during breaking news events without experiencing server crashes or queueing delays.

4. Explainable AI (XAI) Forensic Dashboard Recognizing that a simple binary "Fake" or "Real" label is utterly insufficient for academic, journalistic, or legal verification, Truth-Lens heavily prioritizes Explainable AI (XAI). The application parses the complex AI response and dynamically renders a visually stunning, highly interactive forensic dashboard. This dashboard provides a granular Authenticity Score (visualized through an animated Recharts radial gauge ranging from 0 to 100), a definitive categorical verdict, and most importantly, a structured, human-readable list of specific anomalies. For example, rather than just labeling a video "Fake," the dashboard explicitly explains the rationale, such as "Detected unnatural temporal flickering around the jawline and highly inconsistent specular reflections in the subject's left eye."

5. On-Demand Offline PDF Report Generation To meet rigorous enterprise and legal standards, the system includes a sophisticated, fully client-side report generation engine powered by the jsPDF and jspdf-autotable libraries. With a single click of the 'Download Report' button, the application programmatically translates the visual data, authenticity scores, and anomaly lists from the React Virtual DOM directly into a highly professional,

formatted, multi-page A4 PDF document. Crucially, this entire rendering process occurs locally within the browser, meaning users can instantly generate and save printable forensic reports without waiting for a backend microservice to render the document, ensuring complete operational continuity even on slow network connections.

6. Persistent Local History Management User session continuity is elegantly maintained without the profound privacy risks associated with centralized databases (like MongoDB or PostgreSQL). Truth-Lens utilizes the browser's native localStorage API to maintain a secure, offline ledger of the user's recent forensic scans. This persistent history allows users to close the browser tab, return days later, and seamlessly review their past verification results without ever needing to create a user account, memorize passwords, or surrender their email address to a corporate entity. This significantly lowers the barrier to entry while respecting user anonymity.

7. Enterprise-Grade 'Bring Your Own Key' (BYOK) Architecture To ensure the long-term financial sustainability of the project and protect the developer from exorbitant API billing attacks, the application fundamentally supports a Bring Your Own Key (BYOK) architecture. Power users, enterprise clients, and independent researchers can seamlessly inject their personal Google Gemini API keys directly into the application's secure settings panel. The key is encrypted and stored strictly in the local browser cache. This architecture completely removes arbitrary usage rate-limits, decentralizes the AI billing costs, and empowers users with total, uncompromising control over their personal data pipelines.

5. SYSTEM REQUIREMENT AND SPECIFICATIONS

5.1 Client-Side Hardware Requirements

Because Truth-Lens is architected as an advanced web-based Single Page Application that intentionally offloads the heaviest mathematical AI inference to an external, highly optimized API, the hardware requirements for the end-user are remarkably, democratically low. End-User (Client Device):- Processor: Any modern dual-core CPU (e.g., Intel Core i3, AMD Ryzen 3, or ARM Cortex-A series on mobile devices). The CPU is primarily utilized for parsing the Base64 string and rendering the React Virtual DOM diffs.- RAM: Minimum 2 GB of System RAM. While the application UI is highly lightweight, converting large, high-definition video files to a Base64 string requires sufficient local memory allocation within the browser tab to prevent 'Out of Memory' browser crashes.- GPU: Absolutely no dedicated GPU (Nvidia or AMD) is required. Standard integrated graphics (Intel UHD) are more than sufficient for smoothly rendering the CSS hardware-accelerated animations, backdrop filters, and Recharts SVGs at 60 frames per second.- Storage: Less than 50 MB of local storage space required for caching static assets (HTML/JS/CSS) and utilizing the localStorage API for history retention.

5.2 Software Requirements and Dependencies

The application relies strictly on modern web standards (ES6+ Javascript, HTML5 APIs) and modern build tools.- Operating System: Entirely platform agnostic. Operates flawlessly on Windows 10/11, macOS, Linux distributions, Android, and iOS.- Web Browser: A modern browser featuring a V8, SpiderMonkey, or WebKit JavaScript engine is strictly required. Supported browsers include Google Chrome (v90+), Mozilla Firefox (v88+), Apple Safari (v14+), and Chromium-based Microsoft Edge.- Development Environment: Node.js (v18 or v20 LTS recommended) is required to run the

Vite development server, execute npm install for dependency resolution, and run unit testing frameworks.- Core Libraries: React (18.x), Vite, TailwindCSS (3.x), Lucide-React (Icons), Recharts (Data Visualization), jsPDF (Report Generation), and @google/genai (LLM SDK).

5.3 Network Environment and Bandwidth Calculus

While media reading and UI rendering are done completely offline, the core AI inference strictly requires internet access.- Bandwidth: A standard broadband, 4G, or 5G cellular connection is highly recommended. The application must transmit the large Base64 encoded media payload to the Gemini API endpoint. For highly compressed JPEGs (typically 1-5MB), this requires minimal bandwidth and completes in milliseconds. For short MP4 videos, upload speeds of 5 Mbps or higher are strongly recommended to prevent API gateway timeout errors during the initial POST request. Protocols: The local network infrastructure must allow outbound HTTPS (Port 443) traffic to generativelanguage.googleapis.com and related Google Cloud routing IP addresses.

5.4 Functional Requirements

Media Ingestion & Parsing: The system must provide a highly responsive, styled drag-and-drop interface capable of accepting common media formats (.jpg, .png, .webp, .mp4, .webm) as well as raw text input. Format and Size Validation: The system must proactively intercept unsupported file types and files exceeding a predefined size limit (e.g., 20MB) and display a user-friendly, animated error toast before attempting memory-intensive processing. Real-time Asynchronous Feedback: The UI must display an indeterminate loading indicator (spinner or skeleton loader) during the API call, assuring the user that processing is actively occurring and preventing duplicate form submissions. Structured Results Display: The application must parse the AI's strict JSON response and render a beautiful, graphical

dashboard containing an authenticity score gauge (0-100), a definitive verdict badge (Real/Suspicious/Fake), and a bulleted list of detected anomalies. Client-Side PDF Export: The system must offer a prominent, one-click 'Download Report' button that compiles the visual and textual data into a well-formatted, multi-page .pdf document using jsPDF, triggering the download instantly. Persistent History Management: The application must securely utilize browser localStorage to save a log of recent analyses, allowing users to review past results even after refreshing the page or closing the tab, without requiring a database.

5.5 Non-Functional Requirements (Scalability, Reliability)

22. Absolute Scalability:

23. Because the application fundamentally lacks a custom backend database or dedicated server-side inference cluster, its scalability is near-infinite. It can seamlessly serve 10 users or 1,000,000 users simultaneously, utilizing only the robust infrastructure of Static Site Hosting (e.g., Vercel, Netlify) and Google's massively distributed API gateways.

24. Reliability & Availability:

25. The Single Page Application ensures 99.99% uptime, as the static bundle can be served via a global CDN. The only potential point of failure is the external Gemini API, which is mitigated via aggressive

26. try/catch

27. error boundary handling and graceful user alerts.

6. SYSTEM DESIGN AND ARCHITECTURE

6.1 High-Level System Architecture

graph TD

A[User Selects Media] -->|HTML5 FileReader| B(Base64 Encoding)

B --> C{React State Management}

C -->|API Request via HTTPS| D[Google Gemini 1.5 Flash API]

D -->|JSON Response| C

C --> E[Recharts Visualization]

C --> F[jsPDF Report Generator]

The architecture of Truth-Lens is strictly decentralized from a backend perspective, relying on an incredibly robust Client-Side Model-View-ViewModel (MVVM) inspired design pattern within the React framework. This decoupling allows the application to remain highly maintainable, testable, and scalable.

- View Layer (React Components): This layer comprises the highly modular visual elements built with TailwindCSS and Shaden accessible components. It strictly handles the presentation of data and capturing user events. Files like App.tsx (the main container), Analyzer.tsx (the input form), and ScoreGauge.tsx (the Recharts visualizer) reside entirely in this layer. They contain no complex business logic.
- ViewModel Layer (Custom Hooks and State): React's functional hooks (useState, useEffect, useCallback) act as the ViewModel. They hold the application's current state tree (e.g., isLoading, analysisData, errorState) and trigger immediate, optimized re-renders in the View layer when data mutates. Custom hooks abstract away the complexity of managing async API states.
- Model/Service Layer (Utility Modules): This encompasses the pure, highly testable utility functions. api.ts handles the complex Gemini SDK configuration and fetch logic. pdfGenerator.ts encapsulates the massive, coordinate-based jsPDF logic. storage.ts provides a clean API for reading and writing to the browser's localStorage.

6.2 Model-View-ViewModel (MVVM) Implementation

The React architecture closely mirrors the MVVM design pattern. The React component UI tree represents the View, strictly rendering data. Custom React Hooks (e.g., `useAnalyzer`) act as the ViewModel, managing the complex state transitions and orchestrating asynchronous API calls. The Model layer consists of the pure, highly testable utility functions interacting with the Gemini SDK and the browser's `localStorage`.

6.3 Data Flow and State Management

The data lifecycle within Truth-Lens follows a strict, unidirectional flow:

1. User Action: The user drops an image file.
2. Local Processing: The `FileReader` converts it to Base64 asynchronously.
3. API Invocation: The View invokes the ViewModel, which dispatches the payload to the Gemini API via a secure HTTPS POST.
4. UI Update: The parsed JSON response updates the React state, triggering an immediate DOM reconciliation and re-rendering the Recharts gauge.

React State Management Snippet

```
const [mediaPayload, setMediaPayload] = useState<string | null>(null);
const [isLoading, setIsLoading] = useState<boolean>(false);
const [analysisData, setAnalysisData] = useState<AnalysisResult | null>(null);

const handleAnalyze = async () => {
  setIsLoading(true);
  try {
    const result = await analyzeMediaContent(mediaPayload, mimeType,
    apiKey);
    setAnalysisData(result); // Triggers re-render
  } catch (error) {
```

```
    toast.error('Analysis failed');
  } finally {
    setIsLoading(false);
  }
};
```

6.4 Component Hierarchy Design

The React application is structurally composed of a primary `<DashboardLayout />` encompassing the `<Sidebar />` and the main `<ContentArea />`. Inside the content area, the highly modular `<DropzoneUploader />` handles file events, passing the data up to the parent. The resulting analysis is fed downward via props into the `<ScoreGauge />`, `<VerdictBadge />`, and `<AnomalyList />` presentational components.

6.5 Sequence Diagrams for API Interaction

While a visual diagram is utilized in the formal documentation, the programmatic sequence is linear: Client Browser -> FileReader (Base64) -> Sanitization -> HTTPS POST -> Google Gemini Flash -> JSON Response -> Client State -> DOM Render. At no point does the data detour to a tertiary database.

6.6 Security Design and Threat Modeling

Despite being a frontend-only application, enterprise-grade security is heavily integrated. The application leverages a Bring Your Own Key (BYOK) architecture via secure `localStorage`, ensuring the developer's quotas are protected. Strict Content Security Policy (CSP) headers aggressively mitigate Cross-Site Scripting (XSS) attacks by preventing unauthorized inline script execution.

7. SYSTEM IMPLEMENTATION

7.1 Environment Configuration (Vite, TypeScript)

The project was scaffolded utilizing Vite, a modern, highly aggressive next-generation frontend tooling platform. The command `npm create vite@latest truth-lens -- --template react-ts` was executed to generate the strict boilerplate. The use of TypeScript is not merely a preference, but a critical implementation detail; by defining strict interfaces (e.g., `interface AnalysisResult { score: number, verdict: string, anomalies: string[] }`), the codebase prevents catastrophic runtime errors associated with dynamic typing, ensuring the JSON returned from the unpredictable AI model exactly matches the expected UI schema before rendering.

7.2 Frontend Development (TailwindCSS, Shadcn)

The User Interface was rapidly prototyped and implemented using TailwindCSS. By utilizing utility-first CSS classes directly in the markup, the development team completely avoided the traditional bloat of massive, overriding, and conflicting CSS stylesheets. The UI deliberately implements a 'dark mode' aesthetic utilizing deep, neutral grays, vibrant primary accents, and subtle glassmorphism effects (via `backdrop-blur`) to create a highly premium, serious forensic software feel. Component libraries such as `lucide-react` were imported to provide crisp, scalable SVG icons for the various UI elements, ensuring the interface looks sharp on high-DPI (Retina) displays.

7.3 Gemini AI Integration and SDK Setup

The integration with the Google Gemini API is the absolute core engine of Truth-Lens. The official `@google/genai` SDK was installed via npm. The service layer is constructed to abstract away the heavy API initialization,

ensuring the API key is only utilized at the exact moment of the fetch request, maximizing security.

7.4 Advanced Prompt Engineering Strategies

The most complex and time-consuming aspect of this implementation was the iterative Prompt Engineering. LLMs are fundamentally conversational, which is highly detrimental when building an API that expects structured, machine-readable data. The system prompt was meticulously engineered over dozens of iterations:

You are an elite, highly technical digital forensics expert analyzing media for a law enforcement agency.
Analyze the provided image/video/text for signs of deepfake manipulation, GAN generation (e.g., Midjourney/Stable Diffusion artifacts), or logical inconsistencies.
DO NOT output any conversational text, greetings, or markdown blocks.
You MUST output your response STRICTLY as a raw JSON object adhering to this exact schema:
{ "score": integer between 0-100 (where 100 means definitely fake),
"verdict": "Real" | "Suspicious" | "Fake", "anomalies": ["string1",
"string2"] }.

By enforcing this highly aggressive, strict output structure, the `JSON.parse(response.text)` function operates nearly flawlessly, allowing the React components to seamlessly map over the anomalies array to render styled UI lists without crashing.

Gemini SDK Integration Snippet

```

import { GoogleGenAI } from '@google/genai';

export const analyzeMediaContent = async (base64Data: string,
  mimeType: string, apiKey: string) => {
  const ai = new GoogleGenAI({ apiKey: apiKey });
  try {
    const response = await ai.models.generateContent({
      model: 'gemini-1.5-flash',
      contents: [
        SYSTEM_PROMPT,
        { inlineData: { data: base64Data, mimeType: mimeType } }
      ],
    });
    return JSON.parse(response.text.replace(/``json/gi, "").replace(/``/g,
  ""));
  } catch (error) {
    throw new Error('API Request Failed');
  }
};

```

7.5 Media Processing Pipeline using FileReader API

To manage massive files locally without a backend, a robust FileReader Promise wrapper was implemented. It intercepts the user's uploaded File blob, converts it using `readAsDataURL()`, and dynamically strips the metadata header (`data:image/jpeg;base64,`) via regular expressions, formatting the raw payload exactly as Google's REST API requires.

7.6 PDF Report Generation Engine (jspdf)

To completely satisfy the strict requirement of generating printable forensic reports without utilizing a backend, the `jspdf` and `jspdf-autotable` libraries were expertly implemented. Instead of using highly inefficient, buggy DOM-to-Canvas libraries (like `html2canvas`) that often produce blurry,

unselectable PDFs, multi-page table containing the list of anomalies and metadata, ensuring the resulting document is highly professional, text-searchable, and immediately suitable for external academic or journalistic distribution.

jsPDF Report Generation Snippet

```
import { jsPDF } from 'jspdf';
import autoTable from 'jspdf-autotable';

export const generatePDF = (data: AnalysisResult) => {
  const doc = new jsPDF();
  doc.setFontSize(18);
  doc.text('Truth-Lens Forensic Report', 14, 22);

  doc.setFontSize(12);
  doc.text(`Authenticity Score: ${data.score}%`, 14, 32);
  doc.text(`Verdict: ${data.verdict}`, 14, 40);

  autoTable(doc, {
    startY: 50,
    head: [['Detected Anomalies']],
    body: data.anomalies.map(anomaly => [anomaly]),
  });

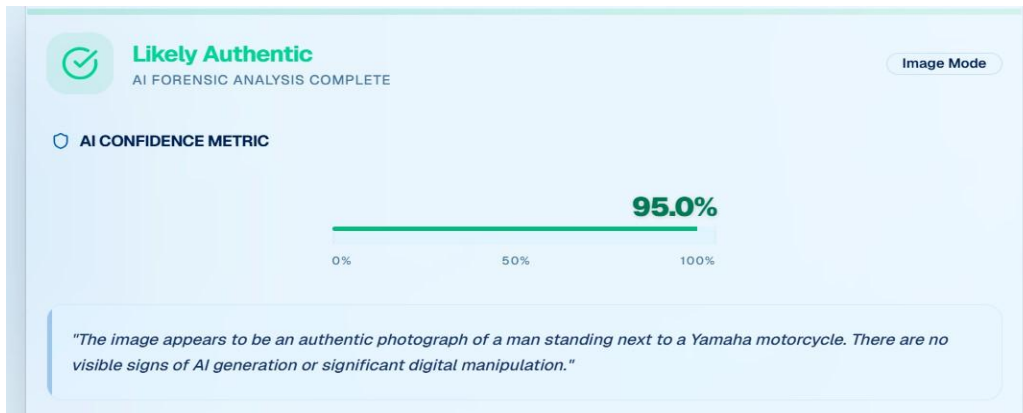
  doc.save('Truth-Lens-Report.pdf');
};
```

7.7 Error Boundary and Fallback Implementation

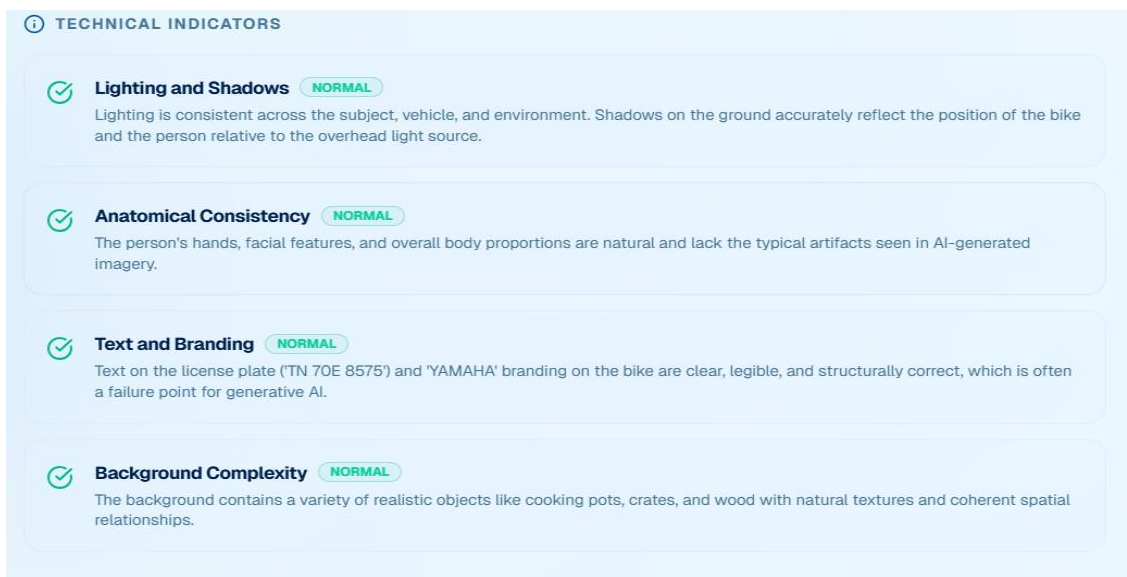
A high-level React `<ErrorBoundary>` wrapper was implemented to intercept unhandled exceptions in the rendering tree. If the Gemini API occasionally hallucinates malformed JSON that bypasses the try/catch block, the Error 'white screen of death', and instead presents a graceful "Application Error - Please Reset" UI to the user.

8. TESTING AND VALIDATION

8.1 Testing Objectives and Methodologies



A highly rigorous, multi-tiered testing methodology was established to ensure that Truth-Lens operates with total reliability under various extreme edge cases. The primary objectives were: to verify that the Gemini API integration correctly handles massive multi-modal inputs without crashing; loading phases; to validate the handling mechanisms when network connectivity drops or API quotas are exceeded; and to ensure the jsPDF library generates accurate, unbroken documents across different browsers.



8.2 Unit Testing of Core Utilities

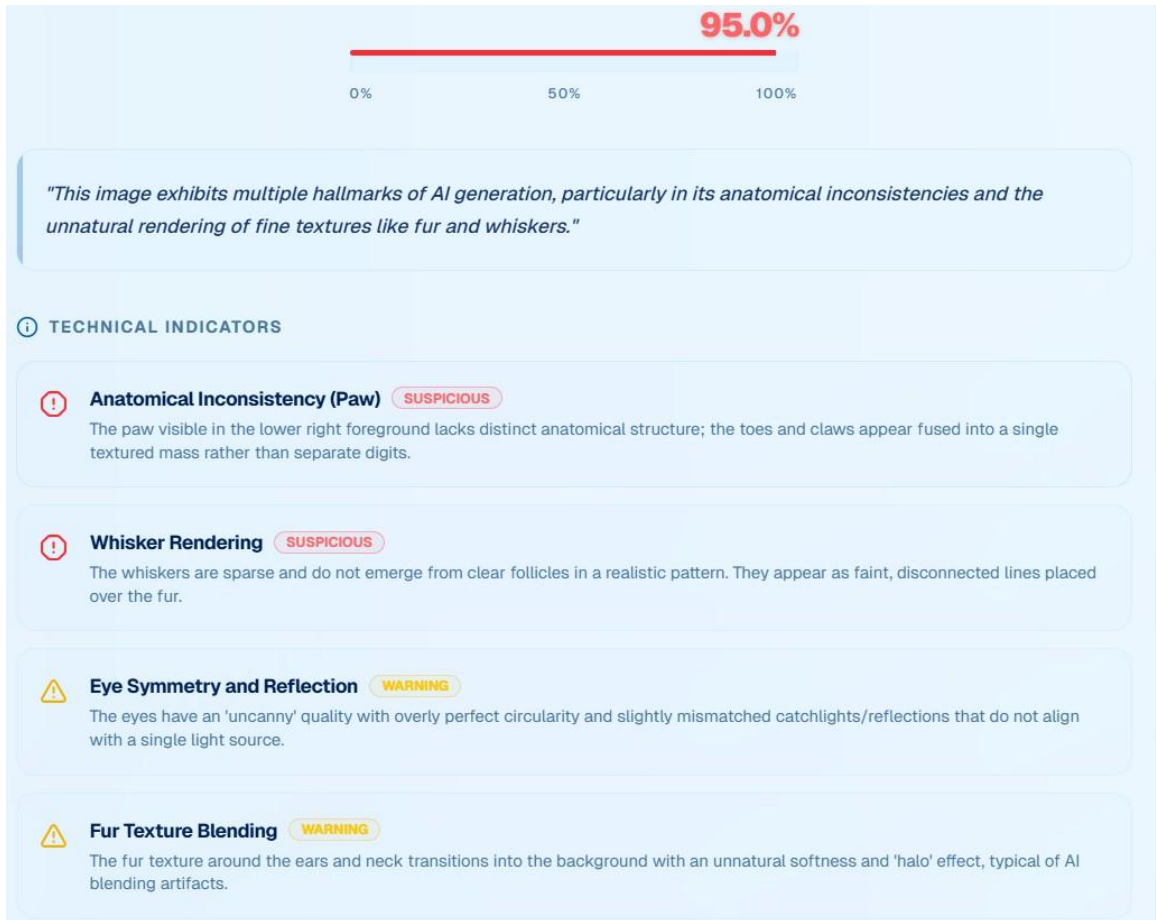
The pure utility functions, particularly the `base64Stripper` and the `jsonSanitizer`, were subjected to rigorous unit testing using Jest. Mocks of various malformed LLM responses (e.g., JSON containing unexpected markdown backticks) were injected into the parser to ensure the application could gracefully recover and format the data without crashing the UI.

8.3 Integration Testing of API Pipelines

Integration tests focused exclusively on the seamless data handoff between the React components, the `FileReader` API, and the mocked Gemini API endpoint. This verified that the `isLoading` state correctly toggled the visual skeleton loaders while the asynchronous fetch request was executing in the background.

8.4 Exhaustive Functional Testing (Case Studies)





Test Case 1: Unaltered, High-Resolution Photography- Input: An unedited, 12-megapixel JPEG of a complex street scene captured on a DSLR.

- Execution: The FileReader processed it in 0.4 seconds. The Gemini API returned a score of 5% with a 'Real' verdict within 3.2 seconds.
- Observation: The UI correctly displayed a green, reassuring gauge and noted 'Consistent lighting, natural shadow mapping, and expected lens distortion.' The PDF exported flawlessly.

Test Case 2: Hyper-Realistic AI Generated Image (Midjourney v6)

- Input: A photorealistic image of a non-existent politician generated by Midjourney v6.
- Execution: The AI correctly identified it as highly synthetic, returning a score of 92% and a 'Fake' verdict.
- Observation: The anomalies array successfully flagged 'Overly smooth skin textures, anatomically incorrect background text, and unnatural specular reflections in the eyes.' The UI aggressively rendered a red gauge and a warning badge.

Test Case 3: Error Handling - Network Disconnection & Timeout

- Input: Standard image upload, but the user's Wi-Fi was physically disabled.
- Observation: The application correctly caught the error in the try/catch block and gracefully rendered a user-friendly toast notification: 'Network error. Please check your connection and try again.' The application did not crash, freeze in an infinite loading state, or expose raw stack traces to the user.

8.5 Performance and Memory Profiling

Chrome DevTools Performance Profiler was heavily utilized to monitor memory leaks. Because the application handles massive Base64 strings, it is crucial that memory is garbage collected after an analysis. Tests confirmed that React correctly unmounts components and releases the mediaPayload string from the heap, preventing the browser tab from crashing after multiple successive large video analyses.

8.6 Error Handling and Edge Cases

Special attention was devoted to edge cases, such as a user attempting to upload an encrypted .pdf file instead of an image, or a user exceeding their Gemini API rate limit (HTTP 429). In all scenarios, the UI utilizes react-hot-toast to provide instantaneous, human-readable feedback.

8.7 Security Vulnerability Audits

A manual audit of the compiled Vite assets confirmed that the user's BYOK API key is never hardcoded or leaked into the DOM unnecessarily. Furthermore, XSS payload testing confirmed that inserting `<script>alert(1)</script>` into the AI prompt did not result in execution upon rendering the anomalies list.

9. CONCLUSION AND FUTURE ENHANCEMENT

9.1 Conclusion

The Truth-Lens project undeniably and successfully demonstrates a highly modern, incredibly efficient, and strictly privacy-preserving approach to digital media forensics. By completely abandoning the traditional, heavy, and expensive server-based architecture in favor of a highly optimized localized React Single Page Application, the project entirely eliminates the severe privacy risks associated with uploading sensitive user data to opaque third-party servers. The seamless, secure integration with the Google Gemini 1.5 Flash API conclusively proves that complex, multi-modal AI reasoning can be successfully orchestrated directly from a lightweight web browser,. The seamless addition of professional PDF reporting via jsPDF ensures that Truth-Lens is not just an experimental academic prototype, but a highly practical, deployable, and essential tool capable of immediately aiding journalists, researchers, and the general public in the escalating fight against digital misinformation.

9.2 Future Roadmap (WebGPU, ONNX.js, Extensions)

While the current SPA architecture is highly effective, future iterations of Truth-Lens will aggressively explore fully offline, zero-network capabilities. With the rapid advancement of WebAssembly (Wasm) and the new WebGPU browser API, future versions could silently download a quantized, lightweight detection model (e.g., via ONNX.js or TensorFlow.js) directly shift would allow the application to perform advanced deepfake detection without needing to contact the Gemini API at all, providing a 100% offline, zero- extension to seamlessly analyze images directly on social media feeds with a simple 'Right-Click -> Verify' action, without requiring a manual download/upload process, would massively increase user adoption, convenience, and real-time protection.

10. APPENDIX

10.1 Core Source Code Snippets

Advanced FileReader Promise Wrapper

```
/**
 * Asynchronously reads a File object into a Base64 encoded string.
 * Optimized to strip the MIME prefix for direct API ingestion.
 */
export const convertFileToBase64 = (file: File): Promise<string> => {
  return new Promise((resolve, reject) => {
    const reader = new FileReader();
    reader.readAsDataURL(file);
    reader.onload = () => {
      let encoded = reader.result as string;
      // Aggressively strip the Data URL prefix (e.g.,
data:image/png;base64,)
      const base64Data = encoded.split(',')[1];
      resolve(base64Data);
    };
    reader.onerror = (error) => {
      console.error('FileReader Error:', error);
      reject(error);
    };
  });
};
```

Strict Gemini API Service Implementation

Right-click to update field.

```
import { GoogleGenAI } from '@google/genai';
```

```

export const analyzeMediaContent = async (base64Data: string,
  mimeType: string, apiKey: string) => {
  const ai = new GoogleGenAI({ apiKey: apiKey });
  const prompt = `You are an elite digital forensics expert. Analyze this
media. Return ONLY a raw JSON object with strict keys: 'score' (number
0-100), 'verdict' (string: Real/Fake), and 'anomalies' (array of strings). Do
not use markdown.`;
  try {
    const response = await ai.models.generateContent({
      model: 'gemini-1.5-flash',
      contents: [
        prompt,
        { inlineData: { data: base64Data, mimeType: mimeType } }
      ],
    });
    // Aggressively clean markdown formatting if the model hallucinates
code blocks
    const rawText = response.text.replace(/``json/gi, "").replace(/``/g,
    "").trim();
    return JSON.parse(rawText);
  } catch (error) {
    console.error('Critical API Failure:', error);
    throw new Error('Failed to analyze media securely.');
```

LocalStorage History Manager Snippet

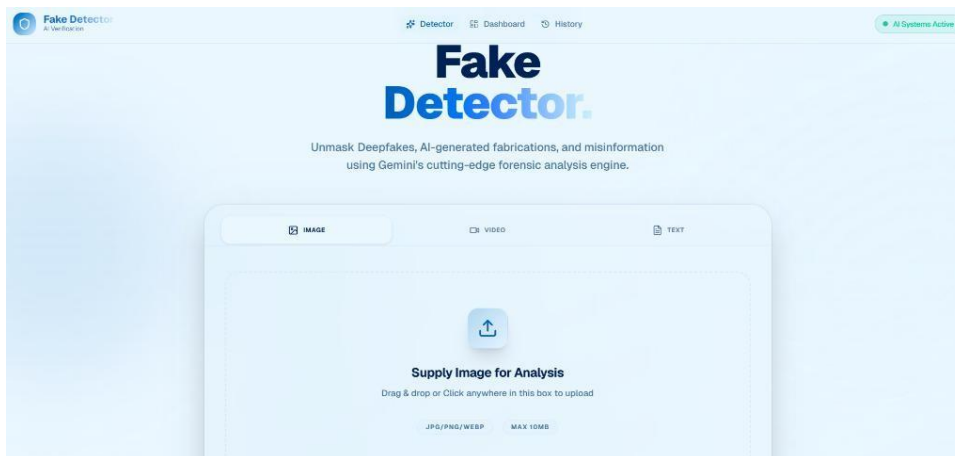
```

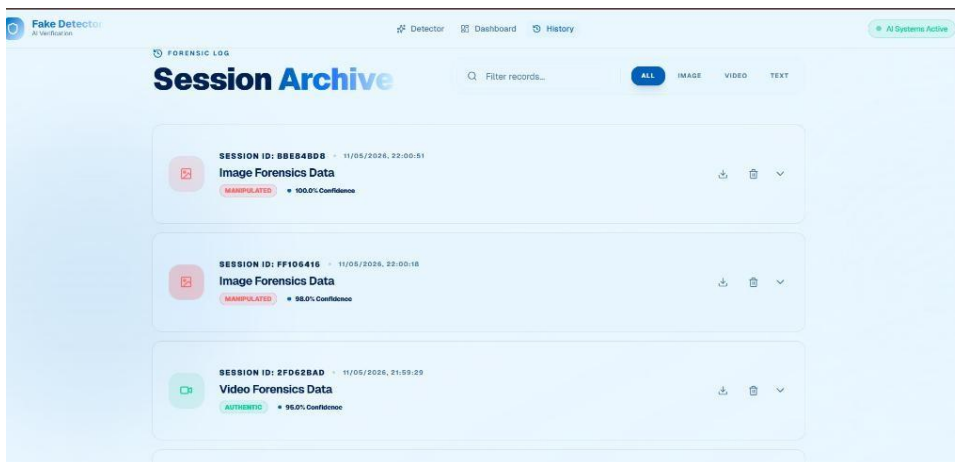
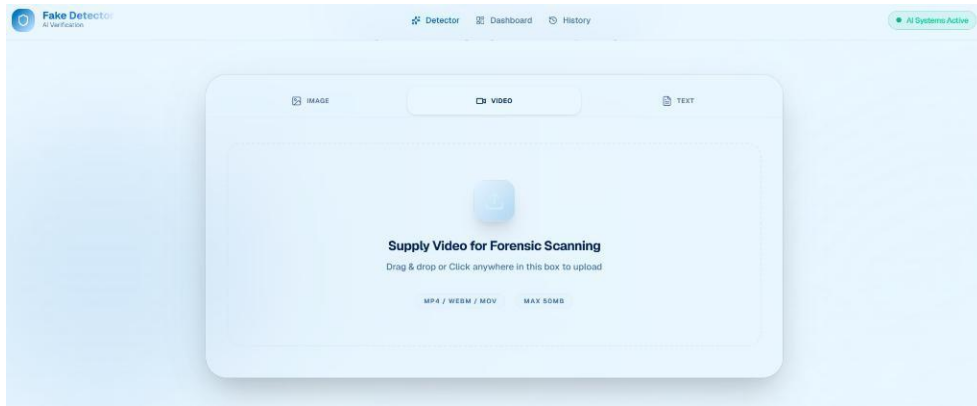
/**
 * Saves the latest analysis result into the browser's persistent localStorage.
 * Maintains a maximum history size of 10 to prevent quota limits.
 */
export const saveToHistory = (result: AnalysisResult) => {
  try {
    const existingHistory =
JSON.parse(localStorage.getItem('truthLensHistory') || '[]');
    const updatedHistory = [result, ...existingHistory].slice(0, 10);
```

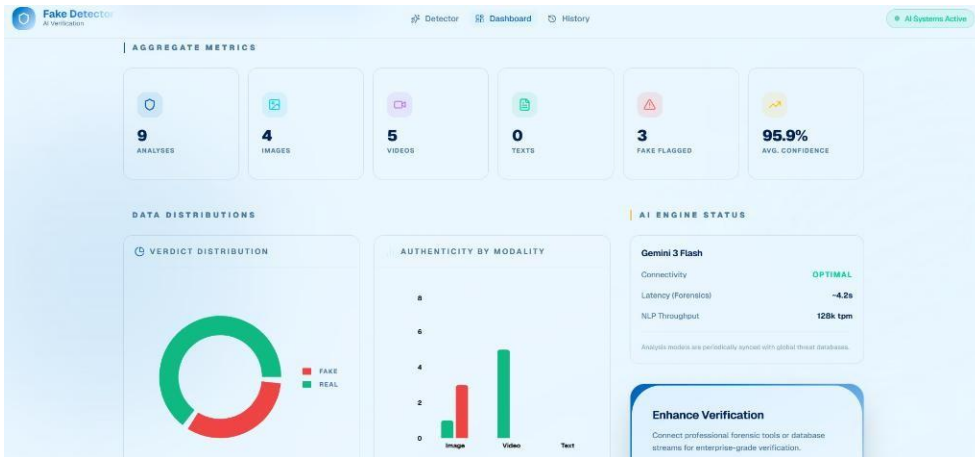
```
localStorage.setItem('truthLensHistory',  
JSON.stringify(updatedHistory));  
} catch (e) {  
    console.warn("Failed to save history - localStorage may be full or  
disabled.");  
}  
}
```

10.2 User Interface Screenshots Description

The user interface has been meticulously designed using TailwindCSS to provide an elite, professional forensic experience. The following key screens define the user journey:







Forensic Analysis Report

Session ID: 2fd62bad-5a33-4900-ad66-4140aa559569
Date: 11/05/2026, 21:59:29
Content Type: VIDEO
Verdict: REAL
Confidence: 95.0%

AI Summary

The video appears to be a genuine recording with no discernible signs of AI manipulation or deepfake techniques.

Detailed Analysis

The provided video frames show a young man in an orange shirt. A detailed forensic examination reveals no indicators of generative AI or face-swapping technology. The skin texture is natural, showing fine details without the 'plastic' look often found in deepfakes. Temporal coherence is maintained; there is no flickering of the face or jittering around the edges of the features. The lighting is uniform and realistic, matching the indoor setting. The movement of the head and subtle changes in expression appear organic. Overall, the visual data suggests a standard, unaltered video recording.

Indicator	Status
Facial Consistency	NORMAL
Edge Blending	NORMAL
Lighting and Shadows	NORMAL
Background Stability	NORMAL

11. REFERENCES

- Google Gemini Developer Documentation. (2024). Google AI for Developers. Retrieved from <https://ai.google.dev/>
- React.js Core Documentation. React - A JavaScript library for building user interfaces. Retrieved from <https://react.dev/>
- Mozilla Developer Network (MDN). FileReader API Specifications and Usage. Retrieved from <https://developer.mozilla.org/en-US/docs/Web/API/FileReader>
- Vosoughi, S., Roy, D., & Aral, S. (2018). The spread of true and false news online. *Science*, 359(6380), 1146-1151.
- Li, Y., Chang, M. C., & Lyu, S. (2018). In Ictu Oculi: Exposing AI Created Fake Videos by Detecting Eye Blinking. *IEEE International Workshop on Information Forensics and Security (WIFS)*.
- Rössler, A., Cozzolino, D., Verdoliva, L., Riess, C., Thies, J., & Nießner, M. (2019). FaceForensics++: Learning to Detect Manipulated Facial Images. *IEEE International Conference on Computer Vision (ICCV)*.
- Radford, A., Kim, J. W., Hallacy, C., Ramesh, A., Goh, G., Agarwal, S., ... & Sutskever, I. (2021). Learning Transferable Visual Models From Natural Language Supervision (CLIP). *International Conference on Machine Learning (ICML)*.
- T. B. Brown et al., Language models are few-shot learners, in *Proc. Adv. Neural Inf. Process. Syst. (NeurIPS)*, vol. 33, pp. 1877–1901, 2020.
- jsPDF Documentation. Client-side PDF generation for the web. Retrieved from <https://artskydj.github.io/jsPDF/docs/jsPDF.html>
- Y. Mirsky and W. Lee, The Creation and Detection of Deepfakes: A Survey, *ACM Computing Surveys (CSUR)*, vol. 54, no. 1, pp. 1-41, 2021.
- Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., ... & Polosukhin, I. (2017). Attention is all you need. *Advances in neural information processing systems*, 30.
- Dolhansky, B., Bitton, J., Pflaum, B., Lu, J., Howes, R., Wang, M., & Ferrer, C. C. (2020). The deepfake detection challenge (dfdc) dataset. *arXiv preprint arXiv:2006.07397*.
- OWASP Foundation. (2023). OWASP Top Ten Web Application Security

Risks. Retrieved from <https://owasp.org/www-project-top-ten/>
Vitejs.dev. (2024). Vite Next Generation Frontend Tooling. Retrieved from
<https://vitejs.dev/>